

Windows-Native LLM 추론 엔진

마이그레이션:

교육용 시스템 아티팩트로서의 micro-vLLM

Windows-Native LLM Inference Engine Migration as an Educational Systems Artifact: micro-vLLM, CUDA Python, Prefix KV Cache, and Green Context Stress Analysis

김 태 원

한국과학기술원 (KAIST) · 서울대학교 (SNU) · (소속 기관)

초 록 고성능 LLM 서빙 시스템은 일반적으로 FlashAttention, Triton, NCCL, vLLM 스타일의 스케줄러, 그리고 생산 수준의 메모리 관리자를 포함하는 Linux-중심 GPU 소프트웨어 스택을 기반으로 개발된다. 이러한 시스템은 효과적이지만, 그 규모와 종속성 가정으로 인해 학생과 Windows 기반 로컬 연구자가 프리필(fill), 디코드(decode), KV 캐시 관리, 스케줄러 결정, CUDA 컨텍스트, 메모리 할당이 추론 엔진 내부에서 어떻게 상호 작용하는지 검사하기 어렵게 만든다. 본 논문은 CUDA Python과 cuTile-스타일 커널 개발 워크플로우를 기반으로 구축된 Windows-native 교육용 추론 엔진 아티팩트인 **micro-vLLM**을 제시한다. 목표는 성숙한 Linux 서빙 스택을 능가하는 것이 아니라, LLM 서빙 메커니즘을 학습하기 위한 간결하고 수정 가능하며 증거를 생성하는 아티팩트를 제공하는 것이다. 아티팩트는 타일링된 MatMul과 융합된 다중 헤드 어텐션(fused multi-head attention)에서 시작하여 최소 LLM 루프 및 nano-vLLM-스타일 서빙 엔진에 이르는 단계적 경로를 따른다. 우리는 완전한 서빙 루프 인과 관계를 가르치는 데 유용한 세 가지 시스템 메커니즘을 평가한다. 첫째, WSL2 FlashAttention은 최적화된 참조 기준선으로 남아 있으며 평균 2138.55 tok/s를 달성한 반면, 현재 Windows-native cuTile 경로는 평균 463.35 tok/s로 4.62배의 처리량 우위를 보인다. 둘째, 시스템 프롬프트, 데이터베이스 스키마, SKILL.md 파일, 또는 강의 컨텍스트와 같은 긴 정적 프리픽스를 반복적으로 재사용하는 고정-컨텍스트 에이전트 워크로드의 경우, 워밍업 프리픽스 KV 캐시 재사용은 계산된 프리필 토큰을 64로 줄이고, 1024, 2048, 3072 토큰 정적 프리픽스에 대해 TTFT를 각각 41.1%, 66.1%, 79.7% 감소시킨다. 셋째, CUDA-core Green Contexts는 RTX 5070 대상 PC에서 32/16 SM 리소스 파티셔닝 기반(substrate)으로 성공적으로 활성화된다. 적대적 반복-프리필 스트레스 워크로드 하에서 Green Contexts는 보호된 디코드 단계 P99 지연 시간을 평균 4.3% 감소시키고 18/20 실행에서 개선을 보인다. 보호된 디코드-갭 P95는 5.0% 개선되고 16/20 실행에

서 개선된다. 그러나 디코드-갭 P99 및 최대 갭은 현재 순차적 엔진 루프에 의해 여전히 지배된다. 종합적으로, 이러한 결과는 추론 엔진 교육이 성공적인 최적화뿐만 아니라 경계가 있는 메커니즘과 부정적 결과를 포함한 전체 서빙 루프를 가르쳐야 한다는 실용적 교육 주장을 뒷받침한다.

키워드: LLM 서빙, CUDA Python, cuTile, 프리픽스 KV 캐시, CUDA Green Contexts, nano-vLLM, 고정-컨텍스트 에이전트, Windows-native GPU 컴퓨팅, 교육용 시스템 아티팩트

1. 서론

로컬 LLM 추론은 강의실, 연구 실험실, 개인 데이터 분석 환경, 오프라인 개발 환경에서 점점 더 중요해지고 있다. 이러한 환경에서 학습자는 단순히 LLM API를 호출하는 것만 필요한 것이 아니라, 프롬프트가 어떻게 프리필 작업이 되고, KV 캐시 블록이 어떻게 할당되고, 디코드가 왜 지연에 민감한지, 스케줄러 결정이 가시적 응답 시간에 어떻게 영향을 미치는지, 그리고 로컬 최적화가 완전한 서빙 루프 내에서 배치될 때 왜 실패할 수 있는지를 이해해야 한다.

vLLM, TensorRT-LLM, FlashAttention 기반 스택, Triton 커널과 같은 생산급 서빙 시스템은 강력한 성능을 제공하지만, 최초의 교육용 아티팩트로는 이상적이지 않다. 이들은 규모가 크고 Linux 지향적이며 최적화된 종속성에 밀접하게 결합되어 있다. WSL2는 Windows 사용자에게 실용적인 경로이지만, 네이티브 런타임과 학습자 사이에 또 다른 경계를 추가한다. Windows-native GPU 개발은 많은 학생과 로컬 연구자가 NVIDIA GPU가 장착된 소비자 Windows 머신을 사용하기 때문에 여전히 중요하다.

따라서 본 논문은 다른 기여, 즉 **교육용 시스템 아티팩트**를 목표로 한다. micro-vLLM은 vLLM의 더 빠른 대체품으로 위치하지 않는다. 학습자와 연구자가 검사해야 하는 서빙 메커니즘, 즉 프리필, 디코드, 페이징된 KV 캐시, 프리픽스 재사용, CUDA 컨텍스트 활성화, 스케줄러 동작, 할당 오버헤드, 테일 지연 시간(tail-latency) 트레이드오프를 노출하도록 설계되었다. 이 아티팩트는 서빙 동작을 생산 스택 뒤에 숨기는 대신 재현 가능한 증거로 전환할 때 가치가 있다.

본 논문의 중심 워크로드는 **고정-컨텍스트 에이전트 워크로드**이다. 많은 에이전트 시스템은 짧은 사용자 요청 앞에 안정적인 컨텍스트를 반복적으로 앞에 붙인다: 시스템 프롬프트, 데이터베이스 스키마, 도구 계약, `SKILL.md` 파일, 정책 텍스트, 예제, 루브릭(rubric), 또는 강의 모듈 등. Text-to-SQL 에이전트는 데이터베이스 스키마와 규칙 컨텍스트를 반복적으로 포함한다. CUDA 튜터링 에이전트는 강의 자료와 커널 템플릿을 반복적으로 포함한다. 교육용 지식 에이전트는 선택된 지식 번들 발췌문을 반복적으로 포함한다. 이러한 워크로드는 프리픽스 KV 캐시 재사용에 자연스럽게 적합하다. 비용이 많이 드는 정적 프리픽스가 요청 간에 재사용될 수 있기 때문이다.

2. 연구 질문

RQ1. Windows-native CUDA Python/cuTile 기반 micro-vLLM 아티팩트는 교육 및 통제된 실험에 적합한 형태로 LLM 서빙의 주요 메커니즘을 노출할 수 있는가?

RQ2. 고정-컨텍스트 에이전트 워크로드에서 프리픽스 KV 캐시 재사용은 Windows-native micro-vLLM 구현에서 계산된 프리필 토큰과 최초 토큰 도달 시간(time-to-first-token)을 얼마나 줄이는가?

RQ3. CUDA Green Contexts 및 동적 형상 패딩(dynamic shape padding)을 포함한 경계가 있는(bounded) 최적화 결과와 부정적 결과는 완전한 서빙 루프 인과 관계에 대해 무엇을 가르치는가?

3. 기여

본 논문은 네 가지 기여를 한다.

1. **Windows-native 교육용 마이그레이션 아티팩트.** micro-vLLM을 MatMul 및 융합 어텐션에서 LLM-from-scratch 및 nano-vLLM-스타일 서빙에 이르는 단계적 학습 경로로 구성한다. 아티팩트는 실제 GPU 동작을 실행하면서도 검사하고 수정하기에 충분히 간결하다.
2. **고정-컨텍스트 에이전트 벤치마크.** 요청이 긴 정적 프리픽스와 짧은 동적 사용자 질문을 공유하는 에이전트-유사 워크로드를 정의하고 측정한다. 이는 추론 엔진 메커니즘을 실제 Text-to-SQL, 튜터링, 지식 에이전트 시나리오와 연결한다.
3. **측정된 프리픽스 KV 캐시 증거.** 웜 프리픽스 캐시는 모든 테스트된 정적 프리픽스 설정에서 계산된 프리필 토큰을 64로 줄이고, TTFT를 최대 79.7%까지 감소시킨다. 프리픽스 변경 부정 제어(negative control)는 0.0% 캐시 적중률과 캐시 없음과 유사한 TTFT를 반환하여 정확한 프리픽스 의존성을 검증한다.
4. **서빙 루프 병목 분석.** 경계가 있는(bounded) 결과와 부정적 시스템 결과를 모두 보고한다: WSL2 FlashAttention은 현재 Windows-native cuTile 백엔드보다 4.62배 빠르며, 동적 형상 패딩은 할당자 압력으로 인해 처리량을 67.04% 감소시키고, CUDA-core Green Contexts는 성공적으로 활성화되어 스트레스 하에서 보호된 디코드 지연을 완만하게 완화하지만 순차적 프리필 유발 일시 중지를 제거하지는 않는다.

4. 배경 지식

4.1 LLM 서빙 루프

자기회귀(autoregressive) LLM 서빙은 **프리필(prefill)** 단계와 **디코드(decode)** 단계로 구성된다. 프리필은 프롬프트를 처리하고 모든 프롬프트 토큰에 대한 KV 캐시 항목을 기록한다. 디코드는 이전에 캐시된 키와 값에 어텐션하면서 한 번에 하나의 토큰을 생성한다. 프리필은 프롬프트 길이와 병렬 계산 처리량에 민감하다. 디코드는 토큰당 지연 시간, 메모리 대역폭, 스케줄러 오버헤드, KV 캐시 접근, 커널 실행(launch) 동작에 민감하다.

이 구분은 한 단계를 개선하는 최적화가 종단 간 서비스 메트릭을 변경하지 않을 수 있기 때문에 중요하다. 예를 들어, 프리픽스 캐시는 프리필 작업과 TTFT를 크게 줄일 수 있지만, 벤치마크가 많은 출력 토큰을 생성하고 디코드가 총 실행 시간을 지배할 때 제한된 처리량 이점을 제공한다.

4.2 페이지징된 KV 캐시와 프리픽스 재사용

페이지징된 KV 캐시는 KV 메모리를 블록으로 나누어 할당, 매핑, 재사용, 제거할 수 있게 한다. 프리픽스 캐싱은 토큰-동일 프리픽스 재사용으로 이 아이디어를 확장한다. 새 요청이 이전 요청과 동일한 토큰 프리픽스를 공유할 때, 런타임은 기존 KV 블록을 재사용하고 접미사만 계산할 수 있다.

이 메커니즘은 정확한 토큰 프리픽스 안정성에 의존한다. 타임스탬프, 실행 ID, 무작위 예제, 또는 프롬프트 초기에 배치된 사용자별 텍스트와 같은 동적 메타데이터는 캐시 적중을 깨뜨릴 수 있다. 따라서 프롬프트 레이아웃은 모델링 문제일 뿐만 아니라 런타임 효율성 문제이기도 하다. 프리픽스 재사용이 원하는 경우 정적 컨텍스트는 동적 사용자 콘텐츠보다 먼저 나타나야 한다.

4.3 CUDA Python, cuTile, nano-vLLM

CUDA Python은 CUDA 런타임 및 드라이버 API에 대한 Python 수준 접근을 제공하므로 호스트 측 GPU 제어를 가르치는 데 유용하다. cuTile-스타일 워크플로우는 타일링된 GPU 커널을 Python 내에서 표현하여 C++ CUDA에 비해 장벽을 낮추면서 고수준 PyTorch 연산자보다 더 많은 하드웨어 동작을 노출한다. nano-vLLM은 vLLM-스타일 서버를 위한 간결한 참조 구현이다. micro-vLLM은 이 교육적 입장을 Windows-native CUDA Python 실험 경로에 적용한다.

4.4 Green Contexts: 리소스 파티셔닝

CUDA Green Contexts는 GPU 리소스, 특히 SM을 컨텍스트 간에 파티셔닝할 수 있게 한다. 본 논문에서는 **리소스 격리 메커니즘**으로 취급하며, 일반적인 속도 향상 메커니즘이 아니다. 관련 질문은 지연에 민감한 디코드 경로가 프리필 간섭 하에서 보호될 수 있는지 여부이다. Green Context 결과는 요청된 리소스-파티셔닝 경로가 실제로 사용되었다는 활성화 메타데이터가 확인된 경우에만 유효하다.

5. 시스템 설계

5.1 설계 목표

micro-vLLM은 네 가지 목표를 중심으로 설계되었다.

- **검사 가능성 (Inspectability):** 학습자는 프롬프트 토큰이 프리필 작업, KV 블록, 디코드 단계로 어떻게 변환되는지 추적할 수 있다.

- **수정 가능성 (Modifiability):** 커널, 스케줄러 로직, 런타임 계측(instrumentation)은 실험 중에 변경하기에 충분히 작다.
- **Windows-native 실행:** 아티팩트는 WSL2를 유일한 실행 경로로 취급하지 않고 Windows CUDA 환경에서 실행된다.
- **에이전트 워크로드 관련성:** 벤치마크는 교육용 에이전트, Text-to-SQL 에이전트, 로컬 지식 에이전트에서 반복되는 정적 컨텍스트를 반영한다.

5.2 마이그레이션 단계

단계	폴더	교육적 역할
0	KernelAgent/0-MatMul	타일링, 공유 메모리, 스위즐링(swizzling), GEMM 기초 학습
1	KernelAgent/1-FMHA	온라인 소프트맥스, 인과 마스크, 융합 어텐션 구현
2	KernelAgent/2-LLM-from-scratch	최소 자기회귀 루프, KV 캐시, CUDA Graph 실험
3	KernelAgent/3-micro-vllm	페이징된 KV 캐시, 프리픽스 캐시, 연속 배치 (continuous batching), Green Contexts

표 1. micro-vLLM 마이그레이션 단계 및 교육적 역할

이 구조는 추론 엔진을 모놀리식 서빙 블랙박스가 아닌 검사 가능한 학습 단계의 시퀀스로 만든다.

5.3 micro-vLLM 서빙 아키텍처

micro-vLLM은 경량 Qwen-계열 서빙 루프를 사용한다. 스케줄러는 대기 및 실행 중인 시퀀스를 관리한다. 블록 관리자는 KV-캐시 블록을 할당하고 해시 기반 프리픽스 재사용을 수행한다. 각 시퀀스는 프롬프트 토큰, 생성된 토큰, 블록 테이블, 캐시된 토큰 수를 추적한다. 모델러너(runner)는 프리필과 디코드 준비를 분리하며, 입력 ID, 위치, 슬롯 매핑, 시퀀스 길이, 블록 테이블을 포함한다.

프리픽스 캐시의 경우, 스케줄러는 시퀀스 할당 중에 완전한 블록 해시를 확인한다. 동일한 프리픽스 블록이 존재하면 `seq.num_cached_tokens`가 증가하고 프리필 경로는 캐시되지 않은 접미사만 모델로 보낸다. 어텐션 컨텍스트는 여전히 전체 키 길이를 반영하므로 모델은 재사용된 프리픽스 KV 블록과 새로 계산된 접미사 KV 블록 모두에 어텐션할 수 있다.

Green Contexts의 경우, 런타임은 요청된 경로가 실제로 활성화되는지 기록한다. 대상 환경에서 PyTorch GreenContext를 가져올 수 있었지만 컨텍스트 생성이 거부되었다. 작동 경로는 `cuda.core`와 `cuda.bindings.driver`, `Device.set_current(ctx)`, `Device.set_current()` 복원을

사용했다. 벤치마크 JSON은 `green_enabled`, `green_api_type`, `green_split_layout_width`, `green_prefill_resource_source`를 기록하여 활성화가 실패할 때 잘못된 성능 주장을 방지한다.

5.4 고정-컨텍스트 프롬프트 레이아웃

벤치마크는 다음 프롬프트 레이아웃을 사용한다.

[정적 시스템 프롬프트]
[정적 정책 / 도구 계약]
[정적 DB 스키마 또는 강의 컨텍스트]
[정적 예제 또는 루브릭]
[동적 사용자 질문]

정적 프리픽스는 요청 간에 의도적으로 동일하다. 동적 접미사는 사용자 질문에 따라 변경된다. 이는 고정 SQLite 스키마에 대한 Text-to-SQL, 고정 모듈에 대한 CUDA 튜터링, 고정 소스 발췌문에 대한 nano-vLLM 튜터링, 안정적인 강의 또는 지식 번들 컨텍스트에 대한 검색 에이전트를 나타낸다.

6. 실험 방법론

6.1 하드웨어 및 소프트웨어

실험은 NVIDIA GeForce RTX 5070 GPU가 장착된 Windows 11 대상 PC에서 수행되었다. 장치는 로컬 아티팩트 메타데이터에서 48 SM, 약 12GB VRAM, 48MB L2 캐시를 보고한다. 최근 Green Context 사전 점검(preflight) 출력은 PyTorch 2.13.0+cu130 및 CUDA 13.0을 보고한다. micro-vLLM은 Qwen-계열 로컬 모델을 사용하며, 스트레스 벤치마크 로그는 대상 PC에서 Qwen3-0.6B를 식별한다.

6.2 기준선 처리량 벤치마크

기준선 처리량 벤치마크는 동일한 동적 다중-사용자 벤치마크에서 Windows-native cuTile 경로를 WSL2 FlashAttention 참조 경로와 비교한다. 이 비교는 Windows cuTile의 우월성을 보여주기 위한 것이 아니라, 성숙한 최적화 스택에 대한 아티팩트의 기준점을 제공한다.

6.3 프리픽스 캐시 벤치마크

프리픽스 캐시 벤치마크는 `KernelAgent/3-micro-vllm/bench_prefix_cache.py`를 사용한다. 각 정적 프리픽스 길이에 대해 벤치마크는 세 가지 조건을 실행한다.

조건	설명
no_cache	각 요청 전에 영구 해시 테이블을 지우고 전체 프리필 수행

조건	설명
warm_cache	캐시를 준비한 후 동일한 정적 프리픽스 재사용
prefix_changed	프리픽스를 변경하여 정확한 프리픽스 캐시 적중을 붕괴

표 2. 프리픽스 캐시 실험 조건

정적 프리픽스 길이는 1024, 2048, 3072 토큰이다. 동적 접미사는 64 토큰이고 생성 길이는 64 토큰이다. 벤치마크는 캐시 적중률, 캐시된 토큰, 계산된 프리필 토큰, TTFT, 종단 간 지연 시간, 디코드 ITL, 처리량을 기록한다.

6.4 Green Context 스트레스 벤치마크

Green Context 스트레스 벤치마크는 `KernelAgent/3-micro-vllm/bench_green_stress.py`를 사용한다. 하나의 보호된 디코드 요청을 활성 상태로 유지하고 대규모 프리필 요청을 반복적으로 주입한다. 주요 메트릭은 **보호된 디코드 완료 갭(protected decode completion gap)**으로, 현재 순차적 엔진 루프에서 가시적 디코드 토큰 사이의 프리필 유발 일시 중지를 포함한다.

파라미터	값
보호된 디코드 프롬프트	32 토큰
보호된 디코드 출력	256 토큰
간섭 프리필 프롬프트	3072 토큰
간섭 프리필 출력	1 토큰
프리필 주입 횟수	12
주입 간격 (디코드 단계)	매 8 디코드 단계 (4 단계 후 시작)
Green 분할	프리필 32 SM, 디코드 16 SM

표 3. Green Context 스트레스 기본 워크로드

Green-측 서버프로세스는 유효한 개입으로 인정받기 위해 `green_enabled=true` 및 `green_api_type="cuda_core"`를 보고해야 한다.

7. 결 과

7.1 WSL2 FlashAttention 참조 대 Windows cuTile

동적 다중-사용자 벤치마크는 133,966개의 생성된 토큰을 처리했다.

백엔드	실행 횟수	평균 시간 (s)	평균 처리량 (tok/s)	상대 처리량
Windows cuTile	3	289.16	463.35	1.00x
WSL2 FlashAttention	4	62.65	2138.55	4.62x

표 4. WSL2 FlashAttention 대 Windows cuTile 처리량 비교

결과는 현재 Windows-native cuTile 백엔드가 성숙한 WSL2 FlashAttention 경로보다 느리다는 것을 보여준다. 이는 논문 프레이밍의 약점이 아니라, 기여가 생산 서빙 대체가 아닌 교육적 마이그레이션, 검사 가능성, 병목 분석으로 위치하는 이유이다.

7.2 고정-컨텍스트 에이전트 워크로드를 위한 프리픽스 KV 캐시

정적 프리픽스	프롬프트 토큰	웹 캐시 적 중률	프리필 (캐시 없음)	프리필 (웹)	프리필 감소	TTFT (캐시 없음)	TTFT (웹)	TTFT 감소	프리픽스 변경 TTFT	종단 간 변화 (웹 vs 캐시 없음)
1024	1088	94.1%	1088	64	94.1%	146.98 ms	86.56 ms	41.1%	148.18 ms	+34.2%
2048	2112	97.0%	2112	64	97.0%	255.58 ms	86.72 ms	66.1%	256.93 ms	+4.9%
3072	3136	98.0%	3136	64	98.0%	381.76 ms	77.47 ms	79.7%	389.61 ms	+1.2%

표 5. 프리픽스 KV 캐시 결과 (정적 프리픽스 1024, 2048, 3072)

웹 프리픽스 캐시는 모든 정적 프리픽스 설정에서 계산된 프리필 토큰을 64로 줄인다. 이는 정적 프리픽스가 완전한 KV 블록으로 재사용되고 64-토큰 동적 접미사만 새로 계산됨을 의미한다. TTFT 감소는 프리픽스 길이에 따라 증가한다: 1024 토큰에서 41.1%, 2048 토큰에서 66.1%, 3072 토큰에서 79.7%.

프리픽스 변경 부정 제어는 0.0% 캐시 적중률과 캐시 없음 조건에 가까운 TTFT를 반환한다. 이는 메커니즘을 검증한다: 개선은 측정 잡음이 아닌 정확한 토큰 프리픽스 재사용에서 비롯된다.

종단 간 처리량은 이 64-토큰 생성 벤치마크에서 일관되게 개선되지 않는다. 이는 디코드 루프가 프리필 이후에도 여전히 총 실행 시간을 지배하기 때문에 예상된 결과이다. 따라서 올바른 주장은 고정-컨텍스트 워크로드에 대한 프리필 작업 및 TTFT 감소이며, 보편적 처리량 개선이 아니다.

7.3 동적 형상 패딩: 부정적 결과

동적 형상 패딩은 JIT 형상 변동을 줄이기 위한 것이었다. 그러나 실제로 각 디코드 단계에서 임시 텐서 할당 및 복사 오버헤드를 유발하여 PyTorch CUDA 캐싱 할당자 압력을 증가시켰다. 패딩 없는 즉시(eager) 경로는 470.82 tok/s를 달성한 반면, 패딩 경로는 155.16 tok/s로 67.04% 처리량 감소를 보였다.

이 부정적 결과는 교육적으로 중요하다. 로컬 최적화 가설은 할당 동작 및 호스트 측 런타임 오버헤드를 포함한 전체 서빙 루프 내에서 평가되어야 함을 보여준다.

7.4 Green Context 활성화 및 비-스트레스 결과

초기 Green Context 로그는 요청된 Green 경로가 자동으로 폴백(fallback)되었기 때문에 유효한 효능 측정이 아니었다. 그 후 런타임은 활성화 메타데이터로 계측되었다. 유효한 cuda-core 실행은 20/20 Green-측 실행에 대해 `green_enabled=true` 및 `green_api_type="cuda_core"`를 기록했으며, 32/16 SM 분할 및 `green_prefill_resource_source="device_sm_fallback"`를 사용했다.

비-스트레스 쌍별 벤치마크에서 유효한 Green 실행은 하나의 기준선 P99 이상치를 고려한 후 안정적인 서빙 수준 이점을 보여주지 않았다. 그 이상치를 제외하면 TTFT는 평균 +0.49%, P99 ITL은 +0.32%, 처리량은 +2.06% 변했다. 이는 일반적인 속도 향상 주장보다는 스트레스 워크로드를 동기 부여한다.

7.5 Green Context 스트레스 결과

반복적인 3072-토큰 프리필 간섭 하에서 cuda-core Green Contexts는 보호된 디코드 평활화를 완만하게 보여준다.

메트릭	기준선 평균	Green 평균	평균 델타	중간 델타	개선된 실행
디코드 단계 P50	50.56 ms	47.37 ms	-5.38%	-5.97%	14/20
디코드 단계 P95	66.33 ms	62.89 ms	-4.83%	-5.41%	13/20
디코드 단계 P99	71.56 ms	68.42 ms	-4.30%	-3.39%	18/20
디코드 갭 P50	51.03 ms	47.89 ms	-5.28%	-5.39%	14/20
디코드 갭 P95	79.45 ms	75.33 ms	-4.98%	-5.77%	16/20
디코드 갭 P99	432.31 ms	426.35 ms	-1.29%	-1.15%	13/20
디코드 갭 최대	441.38 ms	437.94 ms	-0.70%	-0.87%	14/20

메트릭	기준선 평균	Green 평균	평균 델타	중간 델타	개선된 실행
처리량	2098.17 tok/s	2188.79 tok/s	+4.75%	+5.19%	14/20

표 6. Green Context 스트레스: 보호된 디코드 메트릭 (20회 실행 평균)

가장 강력한 Green Context 결과는 TTFT가 아니라, 적대적 프리필 주입 하에서 디코드-단계 테일 평활화이다: 디코드 단계 P99는 18/20 쌍별 실행에서 개선되고, 디코드-갭 P95는 16/20 실행에서 개선된다. 그러나 디코드-갭 P99 및 최대 갭은 기준선에 가깝게 유지된다. 이는 현재 순차적 Python 엔진이 SM 파티셔닝이 완전히 숨길 수 없는 프리필 일시 중지를 여전히 삽입함을 나타낸다. 따라서 Green Contexts는 이 아티팩트에서 **경계가 있는 리소스-격리 메커니즘**이며, 서빙 지연 시간에 대한 완전한 해결책이 아니다.

8. 논의

8.1 고정-컨텍스트 에이전트가 중요한 이유

에이전트 워크로드는 종종 논리적으로 일정한 컨텍스트에 대해 반복적인 프리필 비용을 지불한다. Text-to-SQL 에이전트는 스키마와 규칙 컨텍스트를 반복적으로 포함한다. CUDA 튜터는 강의 자료, 코드 스니펫, 루브릭을 반복적으로 포함한다. 지식 에이전트는 선택된 지식 번들 콘텐츠를 반복적으로 포함한다. 프리픽스 KV 캐시는 이러한 반복 구조를 런타임에 가시적으로 만든다.

따라서 프리픽스-캐시 결과는 프롬프트 구성과 서빙 효율성을 연결한다. 안정적인 컨텍스트는 프롬프트의 시작 부분에 배치되어야 하며, 휘발성 메타데이터는 나중에 이동하거나 캐시된 프리픽스에서 제외되어야 한다. 교육 및 데이터-쿼리 에이전트의 경우, 런타임 인식 프롬프트 레이아웃은 모델을 변경하지 않고도 TTFT를 줄일 수 있다.

8.2 Green Context 결과가 가르치는 점

Green Contexts는 다른 교훈을 제공한다. 이들은 일반적인 가속기가 아니라 리소스-파티셔닝 기반이다. 아티팩트는 세 가지 중요한 공학적 점을 보여준다.

- 첫째, 활성화 유효성을 측정해야 한다. 이전 Green 실행은 런타임이 자동으로 폴백되었기 때문에 무효였다.
- 둘째, 리소스 파티셔닝만으로는 서빙 루프가 순차적일 때 지연 시간 개선을 보장하지 않는다.
- 셋째, 반복적인 프리필 간섭이 있는 스트레스 워크로드 하에서 Green Contexts는 보호된 디코드 지연을 완만하게 완화하지만 프리필 유발 일시 중지를 제거하지는 않는다.

이는 GPU 기능에 지연 시간 변동을 귀속시키기 전에 해당 기능이 활성화되었거나 스케줄러가 올바른 간섭 패턴을 생성한다는 것을 증명해야 하는 일반적인 실수를 방지하는 유용한 교육적

결과이다.

8.3 부정적 결과의 교육적 가치

본 논문은 의도적으로 부정적 및 경계가 있는 결과를 포함한다. WSL2 FlashAttention은 Windows cuTile 경로보다 훨씬 빠르다. 동적 패딩은 처리량을 해친다. Green Contexts는 활성화 메타데이터가 필요하며 현재 엔진에서 경계가 있는 이점만 보여준다. 이러한 결과는 시스템 주장이 어떻게 형성되어야 하는지, 즉 메커니즘, 계측, 제어 조건, 측정, 보수적 해석을 보여줌으로써 아티팩트를 교육에 더 유용하게 만든다.

8.4 제출 범위

본 논문은 더 넓은 ActiveGraph 또는 Tau Coding Agent 프레임워크를 기술적 기여로 포함해서는 안 된다. 해당 주제는 별도의 교육용 지식-에이전트 논문에 적합하다. Paper #1은 micro-vLLM을 추론-엔진 아티팩트로, 그리고 측정된 서빙-루프 동작에 초점을 맞추어야 한다.

9. 타당성 위협

- 실험은 단일 RTX 5070 대상 PC를 사용하며, 데이터센터 GPU나 다른 CUDA 버전으로 일반화되지 않을 수 있다.
- 현재 Windows-native cuTile 백엔드는 WSL2 FlashAttention보다 느리므로, 논문은 생산 서빙 우수성을 주장해서는 안 된다.
- 프리픽스-캐시 이점은 정확한 토큰 프리픽스 안정성에 의존한다. 초기 프롬프트 변동은 캐시 적중을 붕괴시킬 수 있다.
- 프리픽스-캐시 벤치마크는 64개의 생성된 토큰을 사용하므로 디코드가 종단 간 런타임을 지배하고 처리량 이점을 제한한다.
- Green Context 스트레스 결과는 디코드 분할에 대해 대상 `cuda.core` 분할이 하나의 리소스 레이아웃을 반환하기 때문에 `device_sm_fallback`을 프리필 리소스 소스로 사용한다. 아티팩트는 이 메타데이터를 기록하지만, 향후 프로파일링은 정확한 SM 상주 동작을 확인해야 한다.
- Green Context 벤치마크는 여전히 순차적 Python 엔진 루프를 사용한다. 향후 비동기 프리필/디코드 루프는 더 강력한 리소스-격리 주장을 테스트하는 데 필요하다.
- Nsight 수준 카운터(예: L2 적중률, 달성된 점유율, 커널 중첩)는 아직 보고된 증거에 포함되지 않았다.

10. 결론

본 논문은 LLM 추론 엔진 내부를 학습하기 위한 Windows-native 교육용 시스템 아티팩트로서 micro-vLLM을 제시한다. 이 아티팩트는 성숙한 Linux 서빙 스택을 능가하지 않는다: WSL2 FlashAttention은 현재 Windows cuTile 백엔드보다 평균 처리량이 4.62배 높다. 대

신 micro-vLLM은 학생과 연구자가 완전한 서빙-루프 인과 관계를 관찰할 수 있는 검사 가능한 마이그레이션 및 실험 경로를 제공한다.

가장 강력한 측정 결과는 고정-컨텍스트 에이전트 워크로드에 대한 프리픽스 KV 캐시 재사용이다. 웜 캐시는 모든 테스트된 정적 프리픽스 길이에서 계산된 프리필 토큰을 64로 줄이고, 1024, 2048, 3072 토큰 프리픽스에 대해 TTFT를 각각 41.1%, 66.1%, 79.7% 감소시킨다. 프리픽스 변경 부정 제어는 정확한 프리픽스 의존성을 검증한다.

Green Context 실험은 경계가 있는 시스템 통찰력을 제공한다. cuda-core Green Contexts는 RTX 5070 대상 PC에서 성공적으로 활성화되고 반복적인 프리필 간섭 하에서 보호된 디코드 지연을 완만하게 완화한다. 디코드-단계 P99는 평균 4.3% 개선되고 18/20 쌍별 스트레스 실행에서 개선된다. 그러나 디코드-갭 P99 및 최대 갭은 순차적 프리필 삽입에 의해 지배되며, 이는 비동기 서빙 루프 없이는 SM 파티셔닝만으로는 불충분함을 보여준다.

종합적으로, 이러한 결과는 효과적인 추론 엔진 교육이 성공적인 최적화뿐만 아니라 활성화 유효성, 부정적 결과, 워크로드 의존성, 고립된 커널 동작과 종단 간 서빙 동작 간의 구분을 가르쳐야 한다는 논문의 교육적 주장을 뒷받침한다.

참고문헌

- W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," *SOSP* 2023.
- T. Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," *NeurIPS* 2022.
- P. Tillet, H. T. Kung, and D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," *MAPL* 2019.
- NVIDIA Corporation, "CUDA Python Documentation," NVIDIA CUDA Documentation.
- NVIDIA Corporation, "Green Contexts," CUDA Programming Guide.
- GeeekExplorer, "nano-vLLM," GitHub repository.
- vLLM Project, "Automatic Prefix Caching," vLLM Documentation.
- PyTorch Foundation, "torch.cuda.green_contexts: Granular Resource Partitioning for CUDA Kernels," PyTorch Documentation.

부록 A. 원시 증거 파일

프리픽스 캐시:

- KernelAgent/3-micro-vllm/prefix_cache_results_cutile_1024.jsonl

- KernelAgent/3-micro-vllm/prefix_cache_results_cutile.jsonl
- KernelAgent/3-micro-vllm/prefix_cache_results_cutile_3072.jsonl

Green Contexts:

- KernelAgent/3-micro-vllm/green_context_results_cuda_core_32_16_v2.jsonl
- KernelAgent/3-micro-vllm/green_context_stress_cuda_core_32_16.jsonl
- KernelAgent/3-micro-vllm/tests/test_green_contexts_api.py

기준선 및 병목 분석:

- KernelAgent/3-micro-vllm/analysis_report.md
- KernelAgent/3-micro-vllm/test-result-cuTile.md
- KernelAgent/3-micro-vllm/test-result-cuTile-run.md
- KernelAgent/3-micro-vllm/test-result-flash_attn.md
- KernelAgent/3-micro-vllm/test-result-flash_attn-run1.md
- KernelAgent/3-micro-vllm/test-result-flash_attn-run2.md
- KernelAgent/3-micro-vllm/test-result-flash_attn-run3.md
- KernelAgent/paper/gpu_info.txt

※ 본 논문은 한국정보과학회 (KIISE) 논문 양식을 참고하여 작성되었으며, 제출-후보 초안(draft)입니다.